



FileVault

An Android file manager built with Flutter —
browse, organise, clean up, and lock files away in an encrypted
Secure Folder.

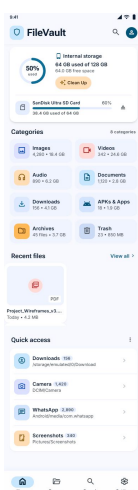
[Project documentation](#) · [Version 1.0.0](#) · [Flutter 3.27+](#) / [Dart 3.8](#)



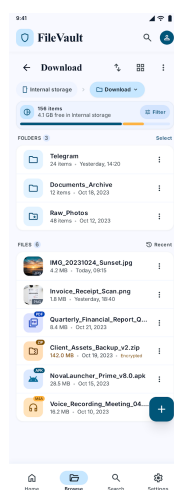
1. Overview

FileVault is a complete file manager for Android phones and tablets, written in Flutter. It does what a file manager is expected to do — browse every storage volume, copy and move files, zip and unzip, search, empty the trash, reclaim space — and adds a Secure Folder: a PIN-protected area where files are encrypted with AES-256-GCM on the device itself and disappear from the gallery and from every other app.

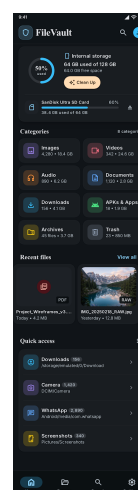
Nothing leaves the phone. There is no account, no sync and no analytics: browsing, search indexing, hashing and encryption all run locally, which is why the app needs no internet permission at all.



Home — storage, categories, quick access



Browser — folders and files



Dark mode — the same screens

About the screenshots

The screenshots throughout this document are the design mock-ups the Flutter UI implements, screen for screen. Build and run the project to see them live on a device.

What is in the box

Area	Capability
Browsing	Internal storage, SD card and USB volumes · breadcrumbs · list and grid · sort by name, size, date or type · hidden files · multi-select
File operations	Copy, move, rename, duplicate, delete, create, share, open with · queued with progress, pause/resume, cancel and conflict handling
Trash	Deleted files are parked on the same volume and restored to their original folder · auto-purge after 7/30/60 days or never
Archives	Create and browse ZIP, TAR, TAR.GZ, BZ2 and XZ · extract selected entries only · password-protected ZIP
Search	Indexed global search with type, size and date filters · in-folder filter · recent searches
Storage tools	Breakdown by category · largest folders and files · duplicate finder (SHA-256) · junk, cache and empty-folder cleaner
Secure Folder	PIN + optional fingerprint · AES-256-GCM encryption · auto-lock · move files in and out
Viewers	Images, video, audio with a queue, text/code editor, PDF and APK details
Organisation	Favourites, recent files, coloured tags, operation history
Appearance	Light, dark and system themes · five accent colours · default layout and sort

Who it is for

Anyone who keeps real files on a phone: someone clearing space before a trip, a student collecting PDFs across folders, a photographer moving a card's worth of RAWs to an SD card, or anyone who simply wants a handful of documents where other apps cannot see them. The common thread is that all of it happens on the device, with no account to create and nothing uploaded.

2. Design language

The interface follows Material 3 with a deliberate, utility-first tone: tonal surfaces instead of heavy shadows, generous touch targets, and colour used to classify rather than decorate.

Type

Roboto Flex carries headlines and screen titles; Inter carries body text, file names and metadata. Sizes, counts and dates are set with tabular figures so columns line up down a long list. Both fonts ship with the app so nothing is downloaded at runtime.

Colour

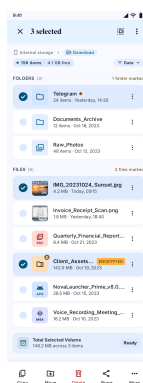
A deep teal-blue primary (#0B6E99) anchors the app, with warm amber (#F2A93B) for pending work and storage warnings. Every file category owns a colour, shown at full strength on icons and at 12–16% opacity behind them:

7C3AED	E1D48	EA580C	2563EB	0B6E99	16A34A	CA8A04	64748B
images	videos	audio	documents	downloads	apks	archives	system

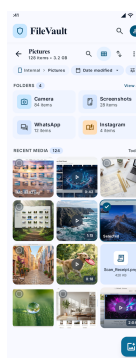
Dark mode is not an inversion: it uses an OLED-friendly carbon canvas (#101418) with stepped surface tiers and 400-level category colours that do not vibrate against dark backgrounds.

Layout

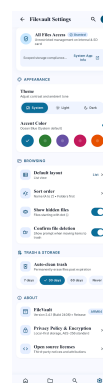
Token	Value	Used for
Margin / gutter	16 dp	Page edges and grid spacing
List row	64 dp	File rows, quick access, settings rows
Touch target	48 dp minimum	Icon buttons, overflow menus, chips
Card radius	16 dp	Cards, grouped lists, grid tiles
Sheet radius	24 dp top	Bottom sheets and dialogs
Grid	3 columns phone 4 on foldables, 5 on tablets	



Multi-select with a running total



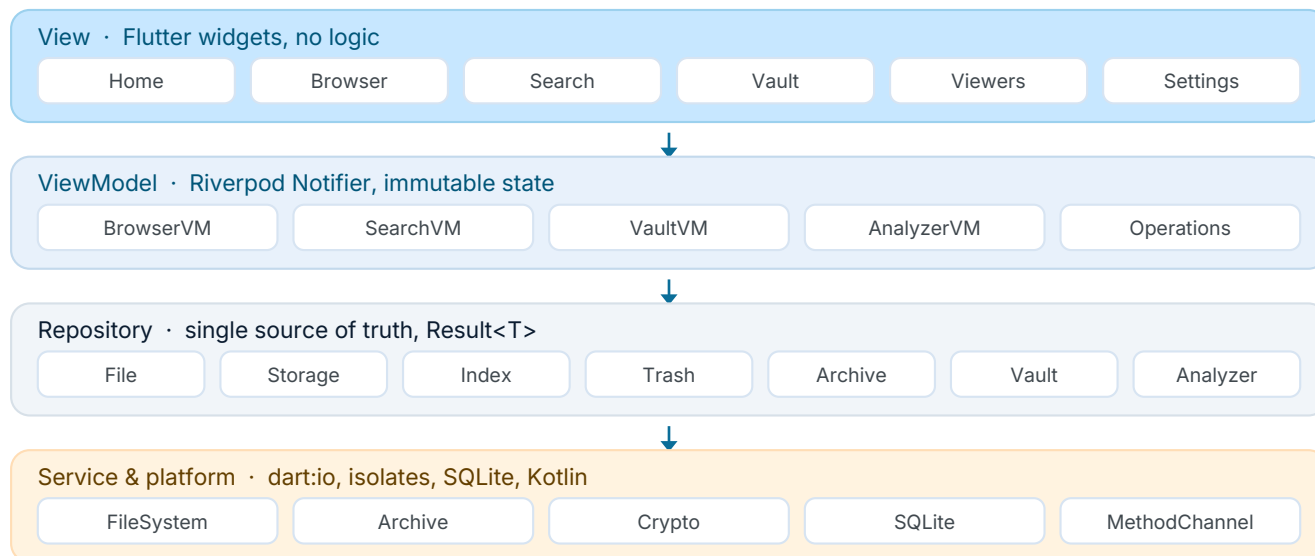
Grid view for media



Settings — theme and accent

3. Architecture

The app is MVVM over a clean, feature-first layering. Dependencies point inward only: a view never reaches past its view model, a view model never imports `dart:io`, and repositories are the single source of truth for everything the UI shows.



Rules the code keeps

- Views hold no logic. They read immutable state and call methods on the view model.
- Errors are values. Repositories return `Result<T>`; a sealed `Failure` hierarchy maps OS error codes (EACCES, ENOENT, ENOSPC, EXDEV...) onto messages the UI can show.
- The file system wins. SQLite only ever indexes and caches; when the two disagree, the directory on disk is right and the row is dropped.
- Nothing heavy on the UI thread. Listing, hashing, archiving, indexing and encryption run in worker isolates with progress and cooperative cancellation.
- No hardcoded strings. Every user-facing word lives in one localisation class.

Project layout

Directory structure

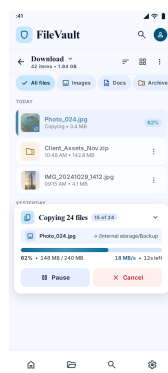
```
lib/
  core/          constants · DI · errors · router · theme · utils · shared widgets
  domain/        models and repository interfaces (no Flutter, no dart:io)
  data/          services (file system, archive, crypto, platform) · SQLite · repositories
  features/      one folder per screen: view + view model + widgets
  l10n/          every user-facing string
android/         manifest · Kotlin platform channel · foreground service
test/           unit and widget tests
```

Choice	What it is	Why
Riverpod	State and dependency injection	Compile-safe providers, easy to override in tests, no BuildContext needed
go_router	Declarative navigation	Tab shell with per-tab stacks, deep links, typed arguments
sqflite	Local database	The index, trash records, tags and vault metadata; ships with Android
archive	ZIP/TAR/GZ/BZ2/XZ	Pure Dart, so archiving runs in an isolate
pointycastle	AES-GCM and PBKDF2	Pure Dart crypto, auditable, no platform gaps
Isolates	Background work	Keeps 60 fps while scanning hundreds of thousands of files

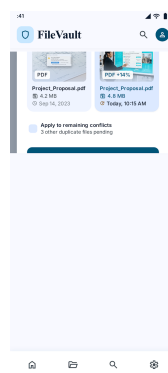
4. File operations

Every long-running action — copy, move, delete, compress, extract, encrypt — goes through one serial queue. The queue survives screen changes, shows a floating progress card above the navigation bar, and posts a foreground-service notification so Android does not kill the work when the app is backgrounded.

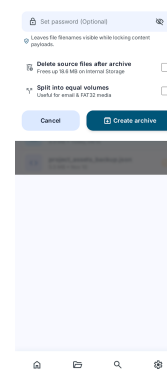
Behaviour	How it works
Progress	Pre-flight pass counts files and bytes, so the percentage, transfer speed and time remaining are real rather than estimated
Pause / resume	A gate that copy loops await between chunks — a paused copy stops mid-file and resumes from the same byte
Cancel	A cancel token checked at every chunk; the partially written destination file is deleted
Conflicts	Replace, Skip or Keep both, with “apply to the rest”; skipped files are never deleted from the source
Same-volume move	Becomes a rename — instant, no copying
Cross-volume move	Copy then delete, file by file, so a failure never loses data
Errors	Permission denied, disk full, path not found and name too long each get their own message



Queue with speed and time left



Conflict — replace, skip, keep both



Create archive

Trash

Deleting moves the file into a hidden `.filevault_trash` folder on the same volume and records where it came from. Restoring puts it back exactly there, or in Downloads if the original folder is gone. Items show a countdown and are purged automatically after the retention period set in Settings.

5. Secure Folder

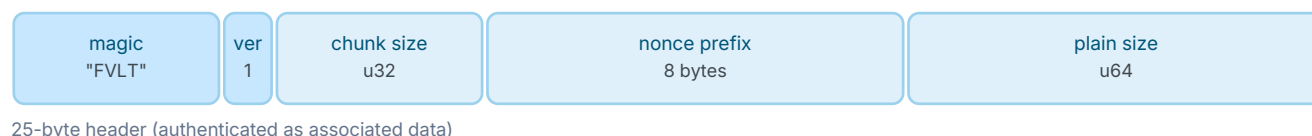
The Secure Folder is the part of FileVault that is not just convenience. Files moved into it are encrypted on the device, stored inside the app's private directory, and removed from their original location — so they no longer appear in the gallery, in other file managers, or over USB.

How the keys work

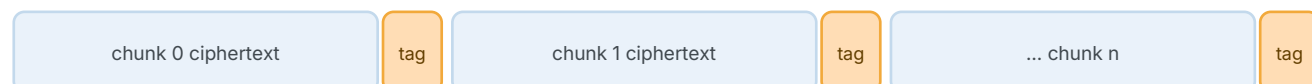
- A random 256-bit master key encrypts the files. It only exists in memory while the vault is unlocked.
- The master key is wrapped with a key derived from the PIN using PBKDF2-HMAC-SHA256, 120,000 rounds, with a random 16-byte salt.
- With fingerprint unlock enabled, the master key is additionally kept in Android's Keystore-backed secure storage and released only after a successful biometric prompt.
- Five wrong PINs trigger a 30-second cooldown. The PIN cannot be recovered — resetting the vault destroys its contents, and the app says so plainly.
- Cloud backup and device-to-device transfer are disabled for the app's files, so the vault never travels off the phone.

File format

Each file becomes a .fv blob with a random name. Content is encrypted in 1 MiB chunks with AES-256-GCM:



1 MiB chunks · AES-256-GCM · nonce = prefix + chunk index



Each chunk's nonce is the file's random 8-byte prefix followed by the chunk index, and the header, the index and a last-chunk flag are authenticated as associated data. That means a chunk cannot be swapped, reordered, duplicated or dropped, and a truncated file is detected rather than silently accepted. Decryption to a temporary file happens only to view an item, and that cache is wiped when the vault locks.

Crash-safety

Moving a file in deletes the original only after the ciphertext is written and the record is stored; if any step fails the whole addition is rolled back, so a file is never lost between the two locations.

Unlocking

The vault locks itself when the app goes to the background — immediately, or after 1, 5 or 15 minutes, whichever Settings says. Opening it shows a six-dot PIN pad with a shake animation on a wrong entry, and a fingerprint key when biometrics are enrolled.

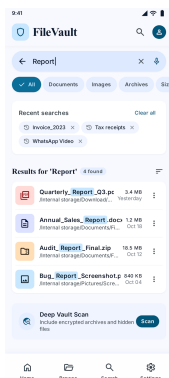
6. Search and storage tools

Search

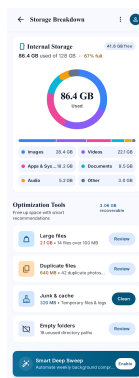
A background scan walks every volume in an isolate and writes file metadata into SQLite. Searching then hits an index rather than the disk, so results appear as you type, with the matched part of the file name highlighted. Filters narrow by category, size bucket and modification date; the in-folder filter in the browser needs no index at all.

Storage analyzer

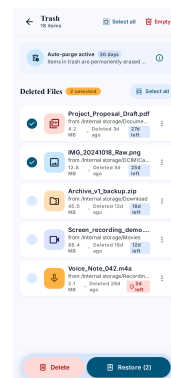
Tool	What it does
Breakdown	Ring chart and segmented bar of space per category, plus the largest folders with file counts
Large files	Everything over 100 MB, newest first, selectable for deletion
Duplicate finder	Groups by size first, then hashes only the candidates with SHA-256 — hashes are cached by path, size and modified time so a rescan is cheap. Keeps the newest copy by default.
Junk & cache	Cache, temp, log, partial-download and thumbnail files, with a preview and a total before anything is deleted
Empty folders	Directory paths with nothing in them, excluding the standard Android media folders



Search with filters and highlights



Storage breakdown and tools



Trash with expiry countdown

7. Android integration

Storage permission

On Android 11 and newer the app asks for All files access (MANAGE_EXTERNAL_STORAGE) through an onboarding screen that explains, in plain words, what it is for. On Android 10 and below it uses the classic storage permission. If the user declines, FileVault still opens — it simply sees its own folders, and the permission can be granted later from Settings. Android/data and Android/obb are detected and explained rather than silently failing.

Kotlin platform channel

Method	Returns
getStorageVolumes	Every mounted volume with name, path, total and free bytes, and whether it is internal, an SD card or USB
getVideoThumbnail	A JPEG frame for video tiles, decoded off the UI thread
getApkInfo	Package name, version, SDK levels, permissions, icon, and whether the app is already installed
scanMedia	Tells the media scanner about new or moved files so galleries update
start/update/finish OperationService	Drives the foreground-service progress notification
openAllFiles AccessSettings	Opens the exact system page for the permission

Permissions declared

AndroidManifest.xml — note there is no INTERNET permission

```
MANAGE_EXTERNAL_STORAGE      all files access (Android 11+)
READ/WRITE_EXTERNAL_STORAGE  legacy storage (capped at API 32 / 29)
READ_MEDIA_IMAGES/VIDEO/AUDIO granular media access (Android 13+)
POST_NOTIFICATIONS           operation progress (Android 13+)
FOREGROUND_SERVICE(_DATA_SYNC) keeps long copies alive
USE_BIOMETRIC                Secure Folder fingerprint unlock
REQUEST_INSTALL_PACKAGES     installing an APK the user taps
```

8. Building and testing

Requirements

Tool	Version
Flutter	3.27 or newer (Dart 3.8+)
Android SDK	compileSdk 35+ · minSdk 26 (Android 8.0)
JDK	17

Getting started

```
unzip filevault.zip && cd filevault
```

```
flutter pub get          # fetch packages
flutter run              # debug build on a connected device
flutter test             # unit and widget tests
flutter analyze          # static analysis
```

```
flutter build apk --release # single APK
flutter build appbundle --release # Play Store bundle
```

Signing a release

Create `android/key.properties` with `storeFile`, `storePassword`, `keyAlias` and `keyPassword`. Without it the release build falls back to the debug key. Minification and resource shrinking are enabled; the keep rules live in `android/app/proguard-rules.pro`.

Tests

Suite	Covers
<code>sort_spec_test</code>	Natural sort (file2 before file10), folders first, every field and direction, input is never mutated
<code>file_utils_test</code>	Category detection, extensions, name validation, unique names, size and duration formatting
<code>file_system_service_test</code>	Listing with and without hidden files, create, rename, duplicate, chunked copy, recursive stats, SHA-256 against a known digest, encoding detection
<code>archive_service_test</code>	ZIP and TAR.GZ round trips, selective extraction, progress reporting
<code>vault_crypto_test</code>	Key wrapping, PBKDF2 determinism, multi-chunk round trip, empty files, and a tampered ciphertext failing the integrity check
<code>search_filter_test</code>	Size and date buckets, match highlighting, trash expiry maths
<code>operations_controller_test</code>	Queue state, progress, speed and time-remaining
<code>widget_smoke_test</code>	File rows, empty states, chips and badges in light and dark

9. Reference

The code in numbers

Dart files	Dart lines	Test lines	Kotlin lines	Screens
107	21,600+	750+	580	20+

Database tables

Table	Holds
file_index	Indexed metadata for global search and the analyzer
hash_cache	SHA-256 per path, size and mtime, so duplicate scans are cheap
trash_items	Original path, trash path, size and deletion time
favorites / recent_files	Bookmarked and recently opened files
tags / file_tags	Coloured labels and their assignments
vault_items	Secure Folder metadata — never the key, never the content
operation_history	Finished operations with status and error
search_history	Recent search terms

Honest limitations

- RAR and 7z are not supported. Both need a native library; the archive engine is pure Dart so it covers ZIP, TAR, GZ, TGZ, BZ2 and XZ. Adding them means a Kotlin channel over junrar / sevenzipjbinding.
- Encrypted ZIP reading depends on the archive package. ZipCrypto and AES entries are handled through it; exotic variants may fail to decrypt.
- SD-card writes on some devices may require Storage Access Framework grants beyond All files access, depending on the manufacturer's ROM.
- Folders cannot be moved into the Secure Folder yet — only files. The format and the queue both support it; the picker does not expose it.
- Only English is bundled. Every string lives in one class, so another language is a subclass and one line in the supported-locales list.

Where to start changing things

Open `lib/core/theme/app_colors.dart` to change the palette, `lib/l10n/app_localizations.dart` to change wording, and `lib/core/di/providers.dart` to swap any repository for a different implementation. Those three files are where most customisation starts.